



CS61C

Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



UC Berkeley
Teaching Professor
Lisa Yan

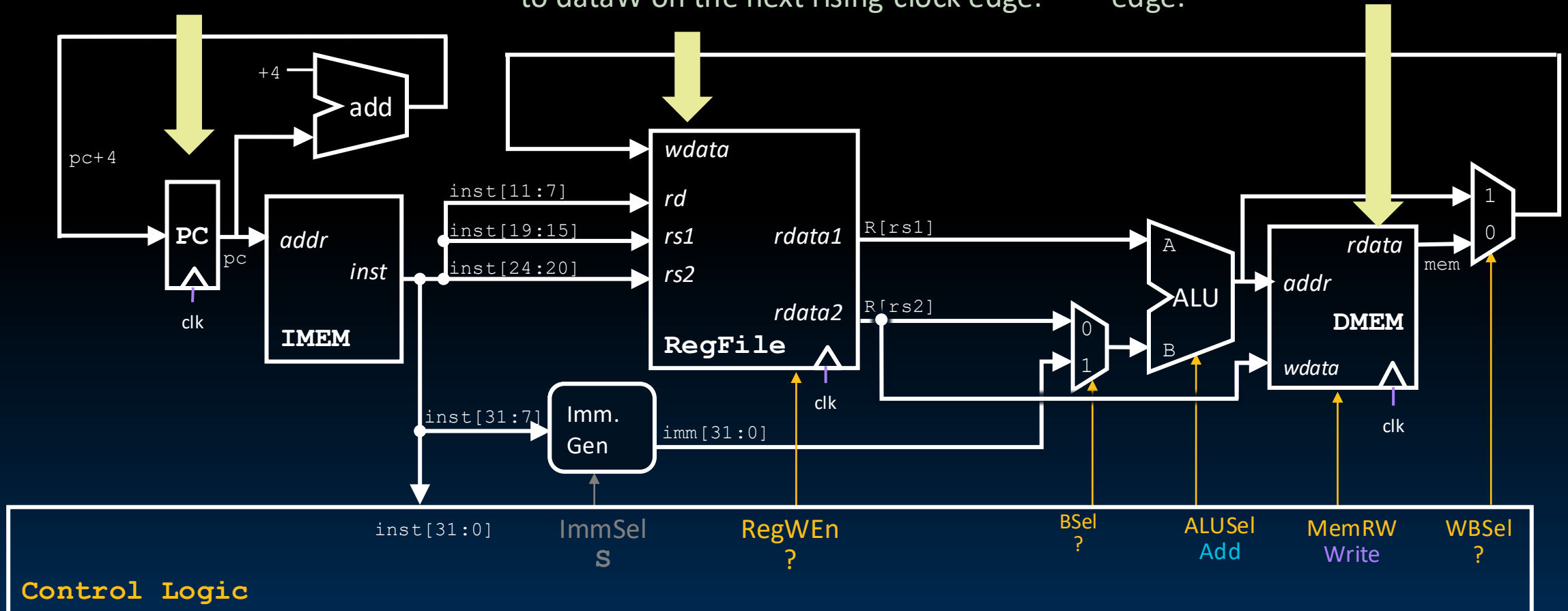
RISC V Datapath II

Our Datapath So Far: Write on rising CLK edge

Rising-edge-triggered write.

If RegWEn=1, RegFile updated with input to dataW on the next rising-clock edge.

If MemRW=Write, DMEM updated with input to wdata on the next rising-clock edge.



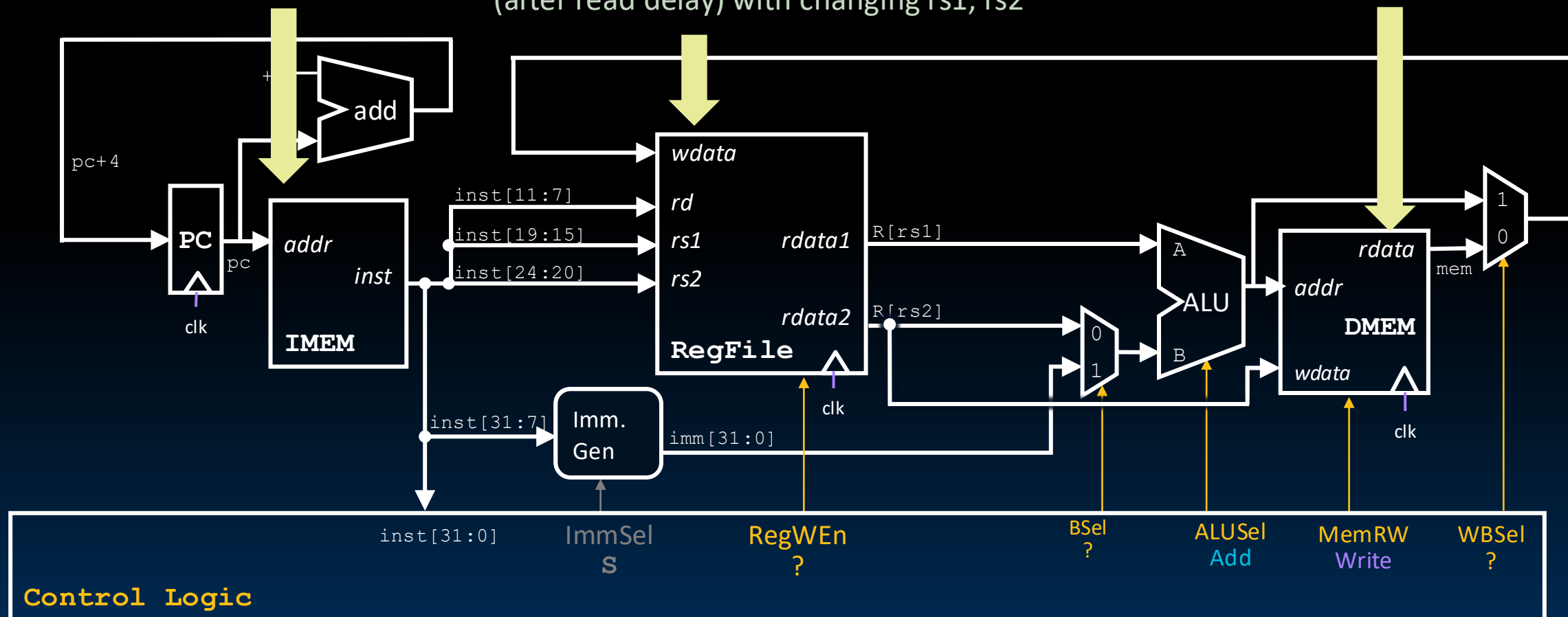
Everything else is combinational logic (CL)

inst updated (after read delay) with changing addr

Read operations are like CL.

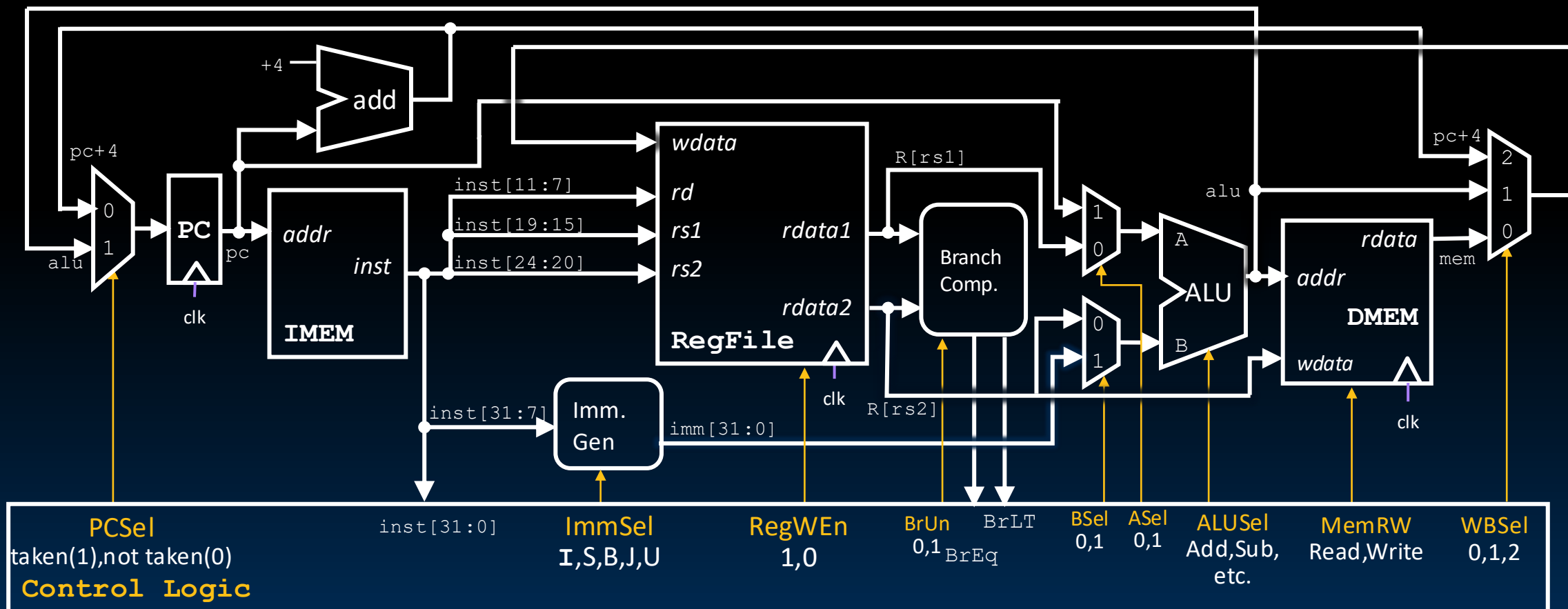
data1, rdata2 outputs continuously updated (after read delay) with changing rs1, rs2

rdata output continuously updated (after read delay) with changing addr



Today: The Complete RV32I Datapath

What do you notice?
New blocks? Additional muxes? New control signals?
What instructions might they be used for?



Yan, SP26



R-Type, I-Type, S-Type

See reference card on [course website](#).

For each instruction below:

- Which wires should hold relevant data, i.e., what is the “lit up” datapath?
- What should the control signals be set to?
- Does this instruction use all 5 phases?
- What is this instruction’s **critical path**?

1. `add rd rs1 rs2`

2. `lw rd imm(rs1)`

3. `sw rs2 imm(rs1)`



Yan, SP26

R-Type: add rd rs1 rs2

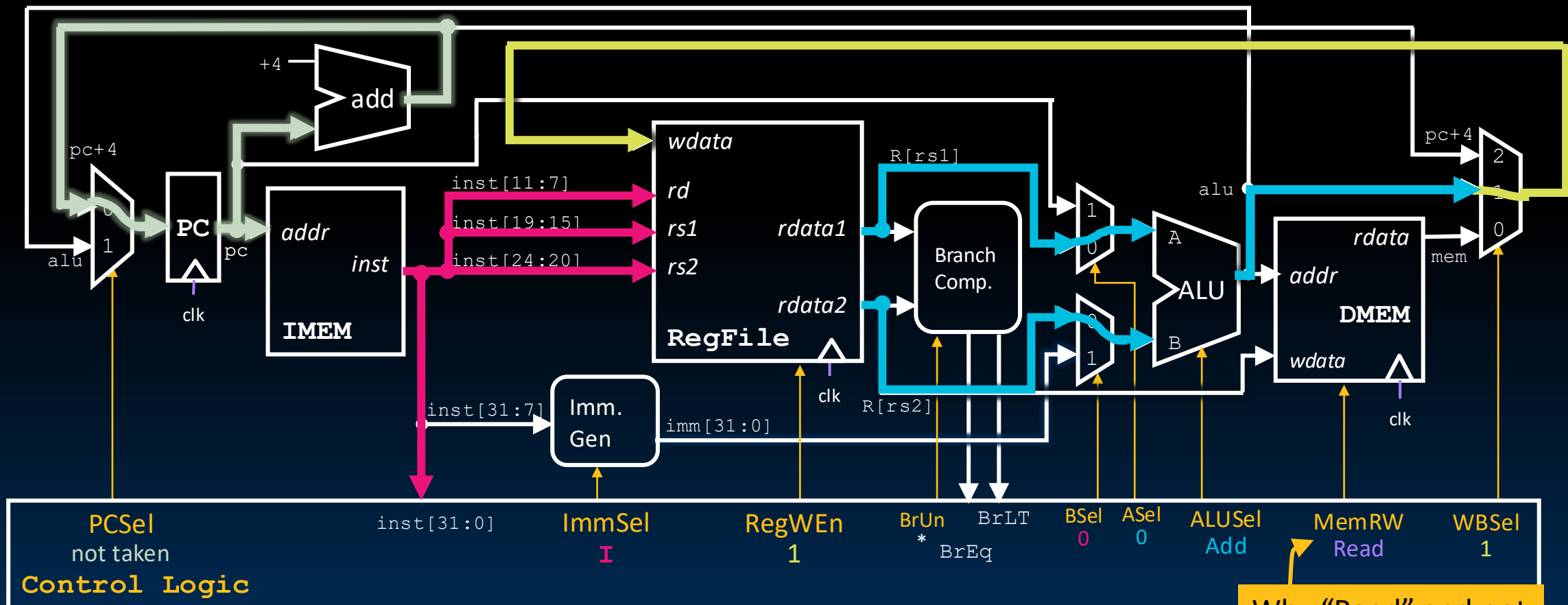
Increment PC to next instruction.

Read from register file.

ALU computes
address $alu = R[rs1] + R[rs2]$.

Don't write to memory.

Write result alu to register.



Why "Read" and not "Write"?

B-Type: beq rs1 rs2 offset

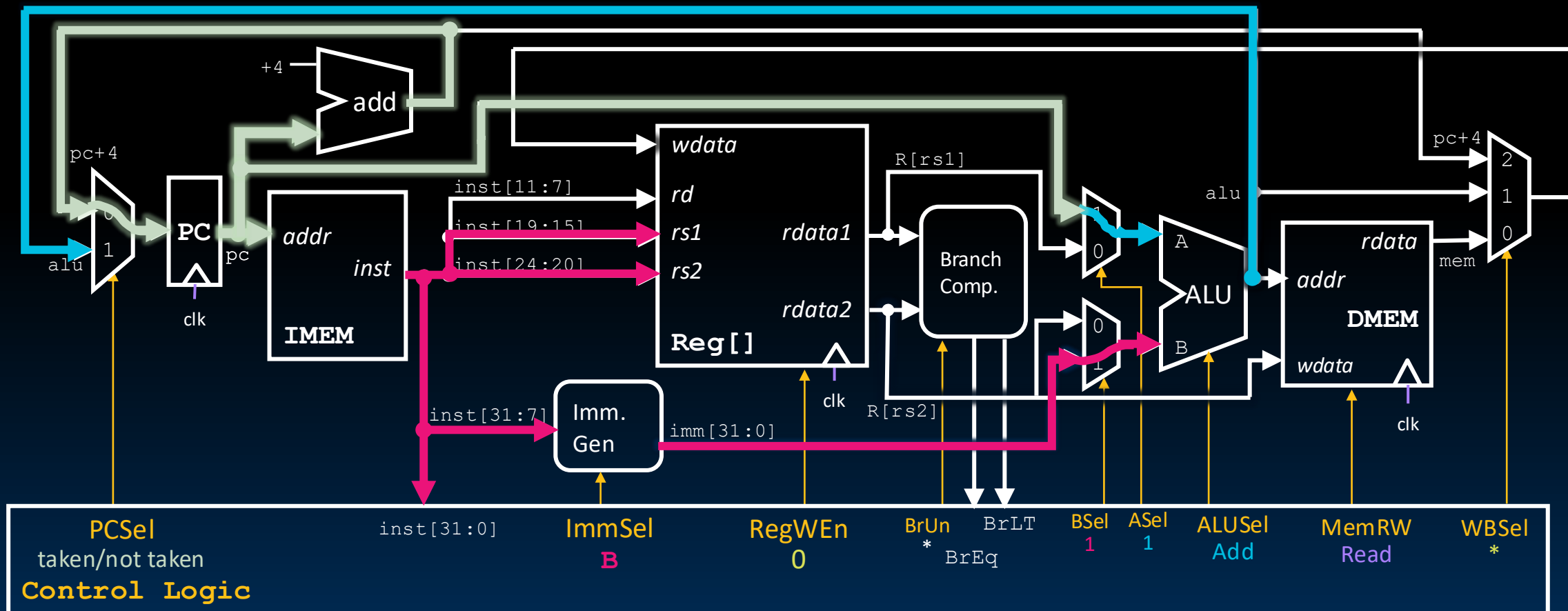
If **BrEq** is 1, update PC to **ALU output**. Else, update to next instruction **PC + 4**.

Build **imm** from B-type instruction.

- Compute branch from **R[rs1]** and **R[rs2]**; feed to **Control**.
- Compute **PC + imm**.

Don't write to memory.

Don't write to registers.



LOAD: lw rd imm(rs1)

Load uses all five stages of the datapath!

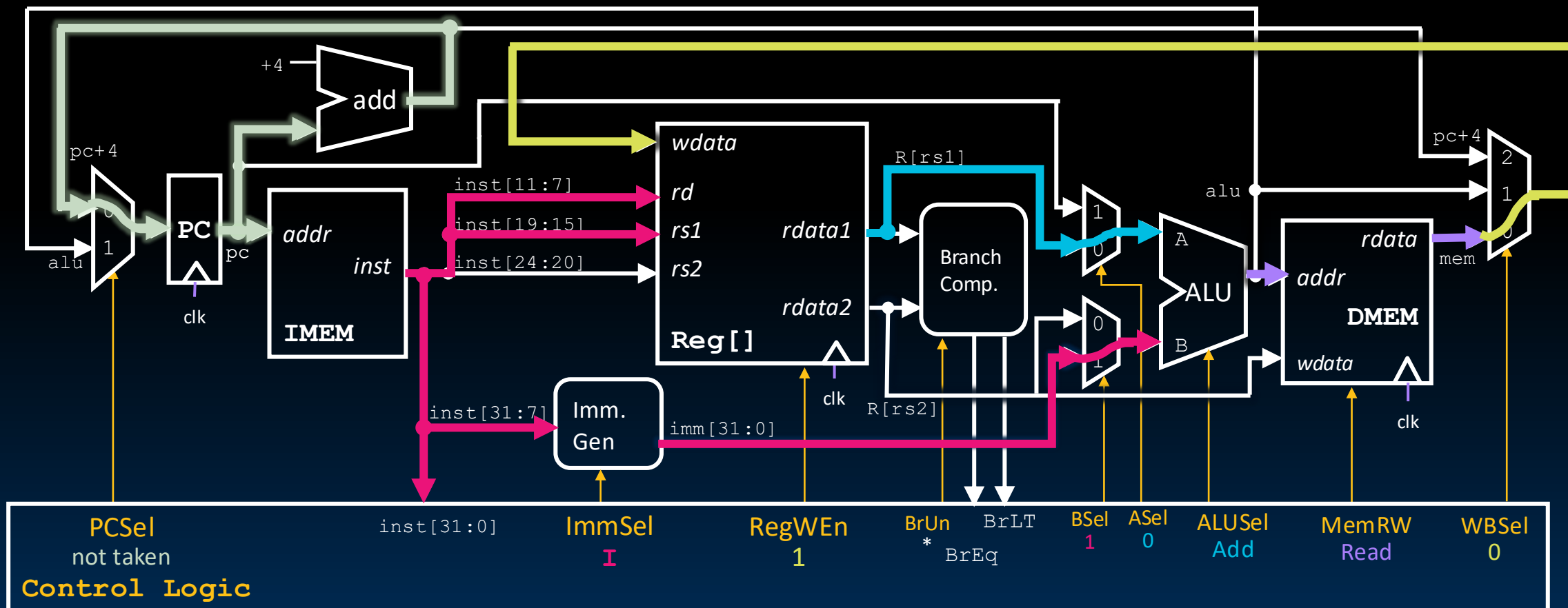
Increment PC to next instruction.

Generate **imm** with upper 12 bits. (I-Type)

ALU computes address **alu** = $R[rs1] + imm$.

Read memory at address **alu**.

Write loaded memory value **mem** to register.



S-Type: `sw rs2 imm(rs1)`

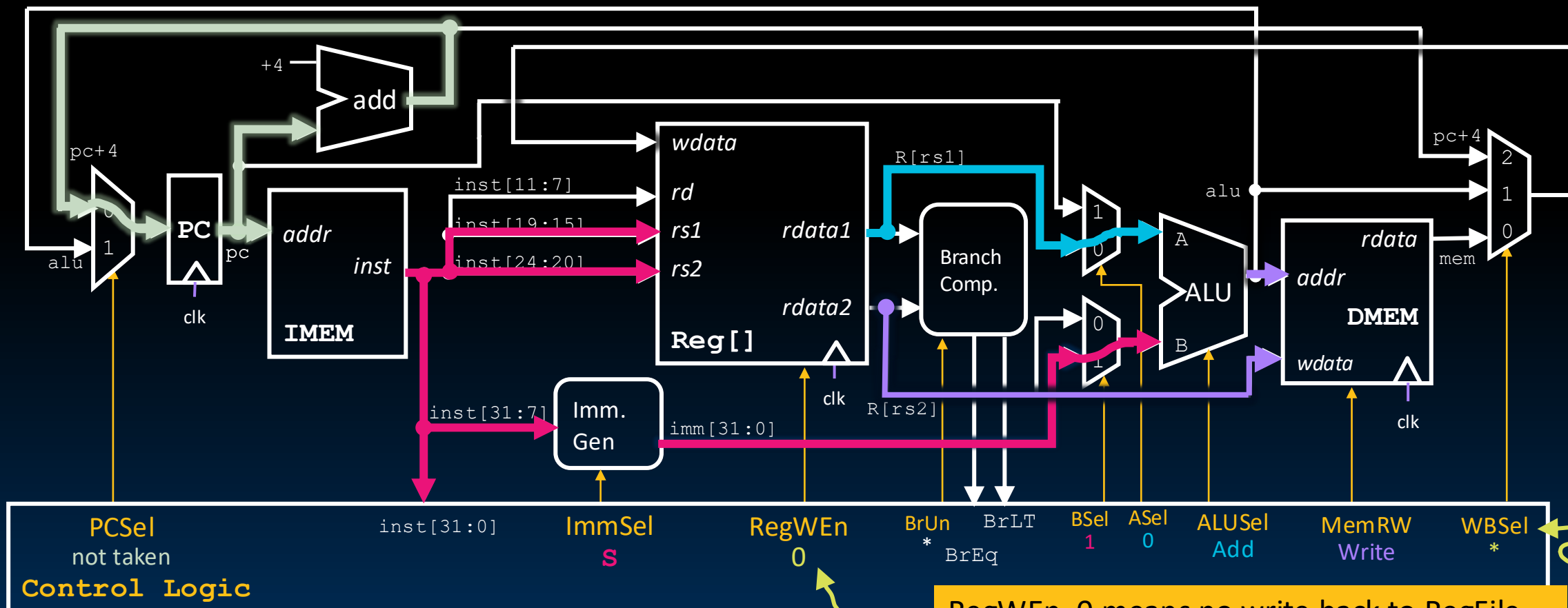
Increment PC to next instruction.

Build **imm** from S-type instruction.

ALU computes address **alu** = $R[rs1] + imm$.

Write memory at address **alu**.

Don't write to registers.



RegWEn=0 means no write back to RegFile, so “*don’t care*” (*) about WBSel’s value.

Adding B-Type (Branches)

- Practice: R-, I-, S-Type
- Adding B-Type (Branches)
- Designing the Immediate Generation Block (cont.)
- [at home] Jumps, U-Type

B-Type: Conditional Branch

- **B-Type** (textbook: **SB-Type**) close to S-Type:
opname rs1 rs2 Label
beq rs1 rs2 offset
- **New Immediate Format!**
- **PC state element now conditionally changes:**
 - RegFile R[rs1], R[rs2] (read only, for branch comparison)
 - PC $PC = PC + imm$ (if branch taken)
 $PC = PC + 4$ (otherwise, not taken)
- **Additionally, need to compare R[rs1], R[rs2]**

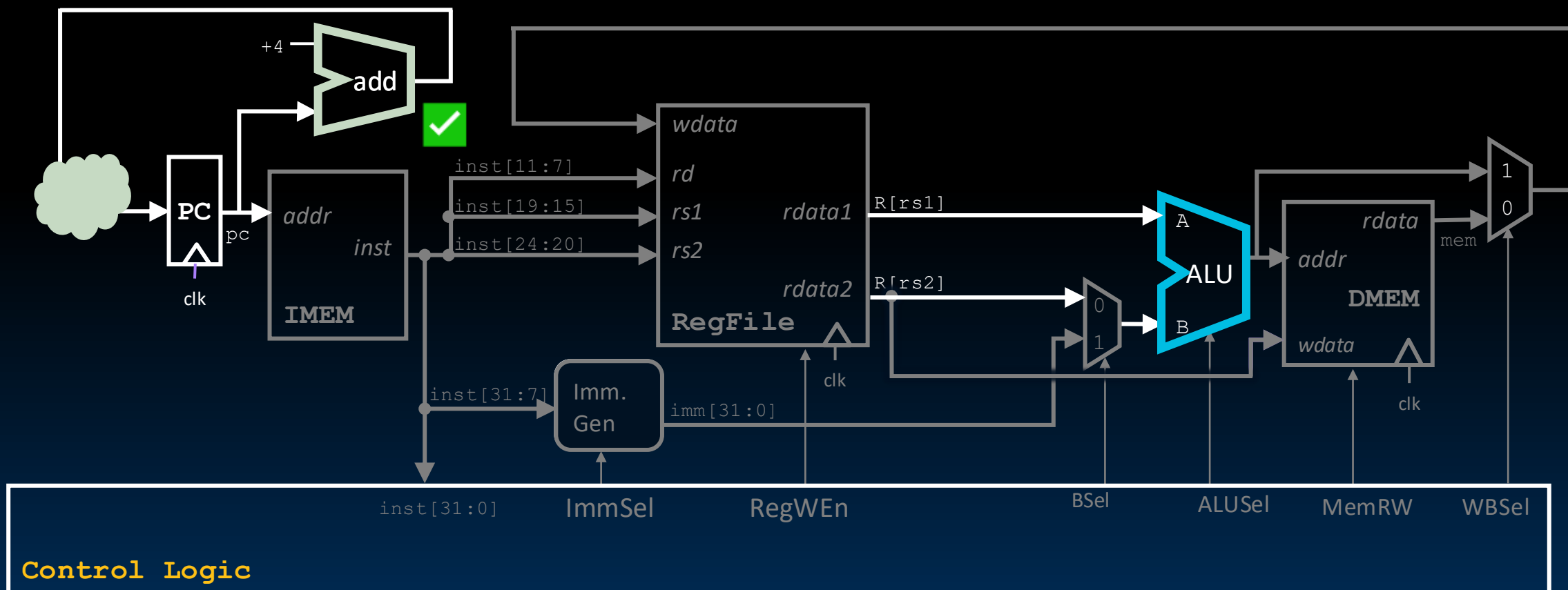
Three arithmetic ops for branches

1. $PC = PC + 4$ ✓

2. $PC = PC + imm$?

3. Compare $R[rs1], R[rs2]$?

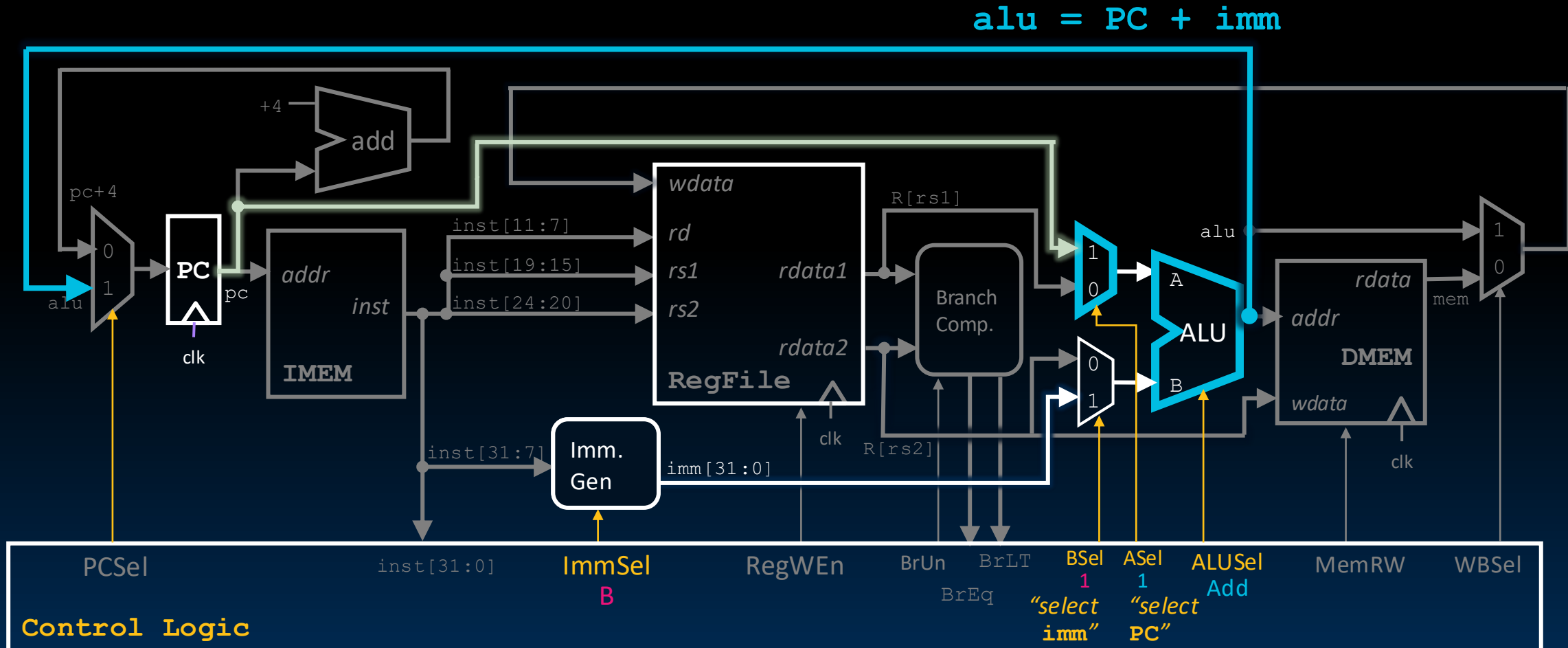
Only one ALU accessible in the same clock cycle. Need more hardware!!



Control Logic

2. The ALU adds PC, Imm

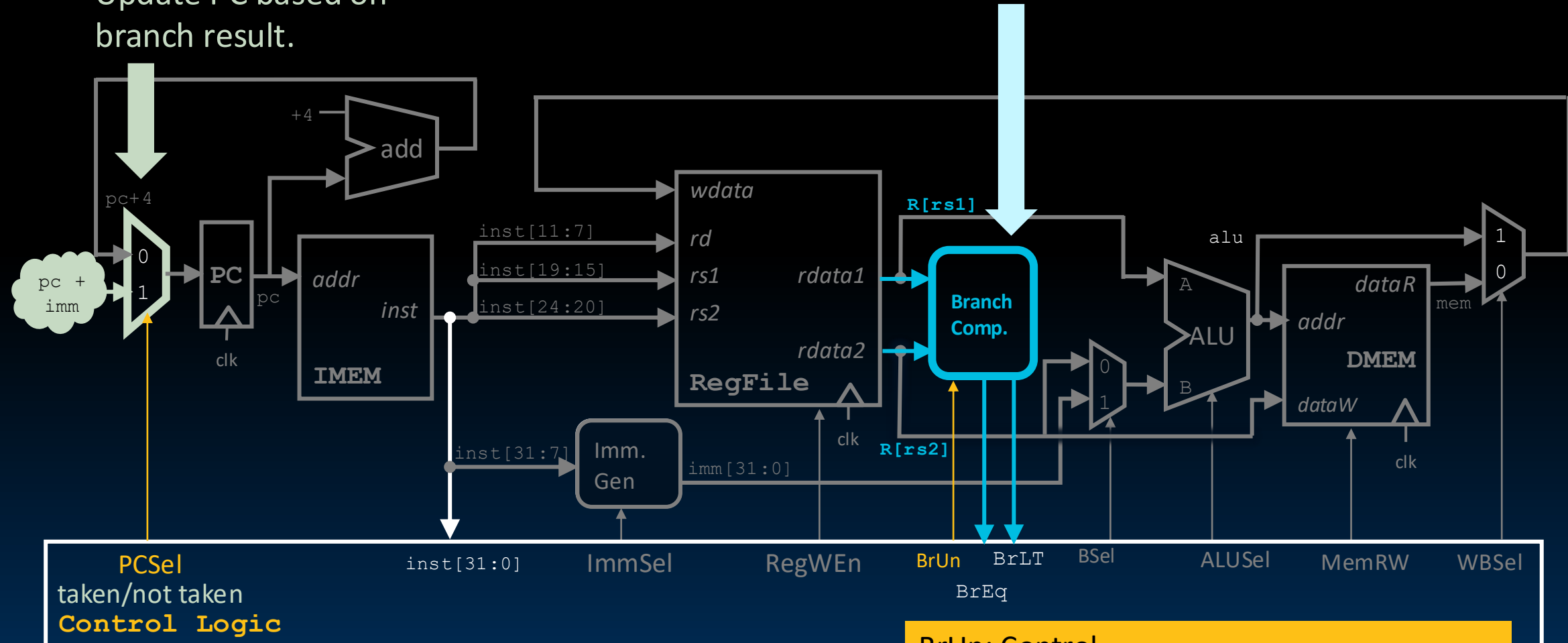
- To compute the address to branch to, use the ALU:



3. The Branch Comparator Block Compares Regs

Compare $R[rs1]$, $R[rs2]$,
feed result to **Control Logic**.

Update PC based on
branch result.



B-Type: beq rs1 rs2 offset

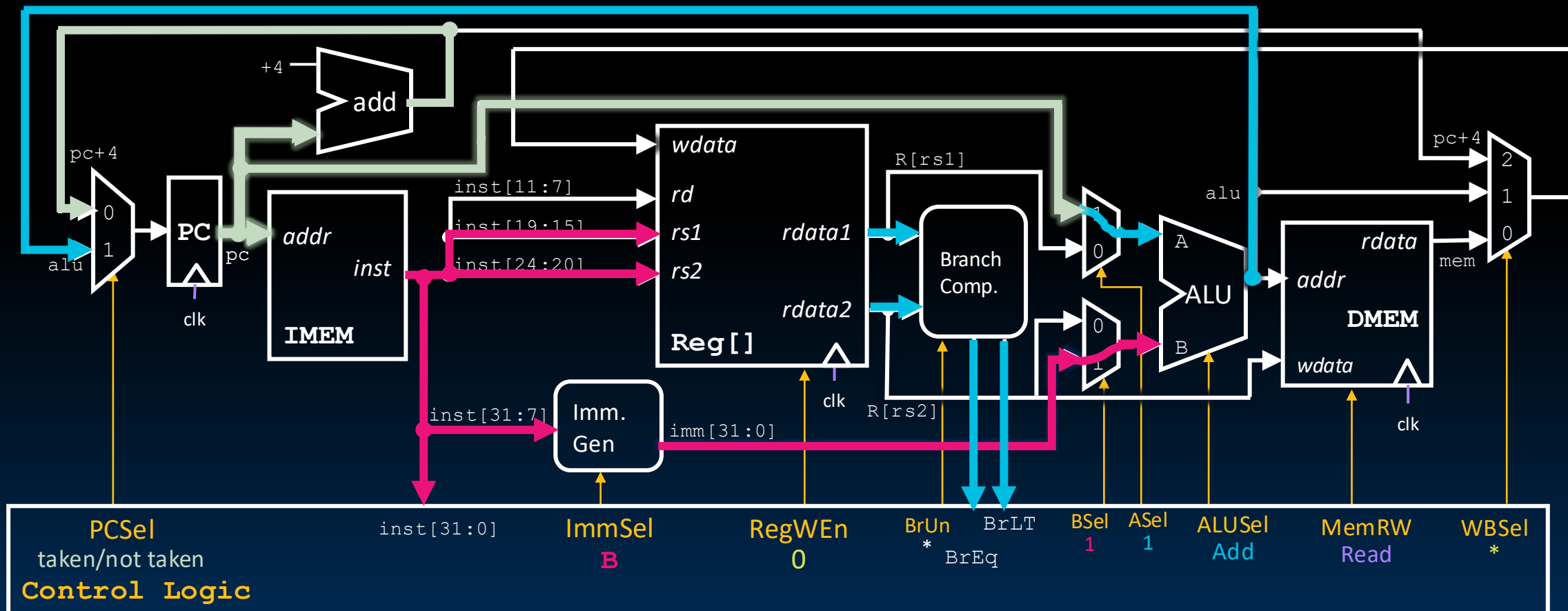
If **BrEq** is 1, update PC to **ALU output**. Else, update to next instruction **PC + 4**.

Build **imm** from B-type instruction.

- Compute branch from **R[rs1]** and **R[rs2]**; feed to **Control**.
- Compute **PC + imm**.

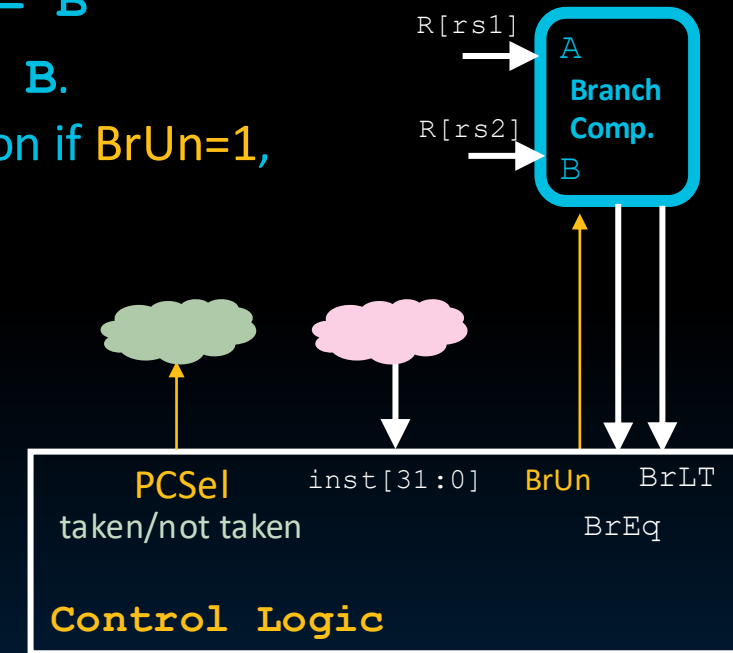
Don't write to memory.

Don't write to registers.



The Branch Comparator Block

- The Branch Comparator is a combinational logic block.
 - Input:
 - Two data busses **A** and **B** (datapath $R[rs1]$ and $R[rs2]$, respectively)
 - **BrUn** (“Branch Unsigned”) control bit
 - Control Logic:
 - Set **BrUn** based on current instruction, $inst[31:0]$.
 - Set **PCSel** based on branch flags **BrLT**, **BrEq**.
 - Examples:
 - **blt**:
 - If **BrLT**=1 and **BrEq**=0, then **PCSel**=taken.
 - **bge**: $(A \geq B) = \overline{A < B}$
 - If **BrLT**=0, then **PCSel**=taken.
- Output:
 - **BrEq** flag: 1 if **A** == **B**
 - **BrLT** flag: 1 if **A** < **B**.
Unsigned comparison if **BrUn**=1, signed otherwise.



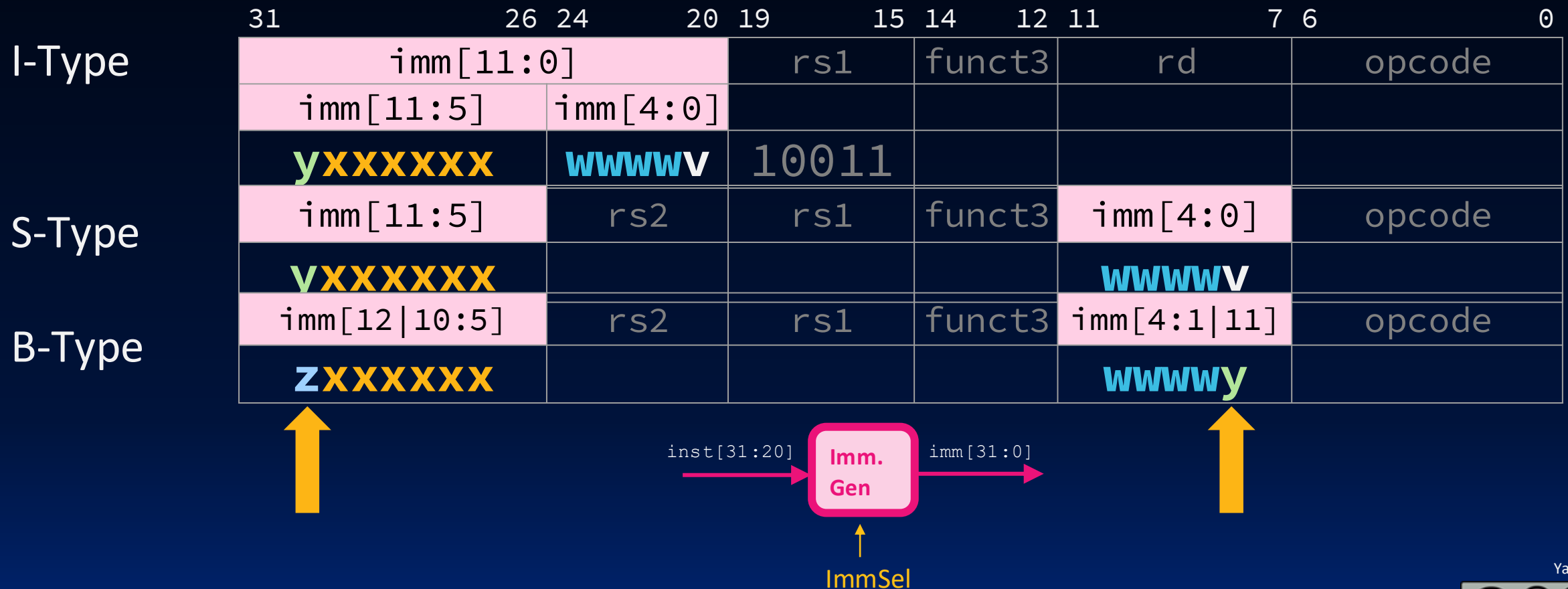
Designing the Immediate Generation Block

- Practice: R-, I-, S-Type
- Adding B-Type (Branches)
- Designing the Immediate Generation Block (cont.)
- [at home] Jumps, U-Type



[Review] Swirl Immediate Bits around for HW Design

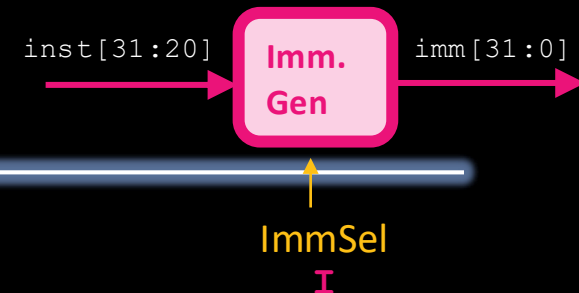
- Recall RISC-V keeps read/write register fields consistent across instruction formats.
- RISC-V also tries to keep **bit positions of immediates** consistent:





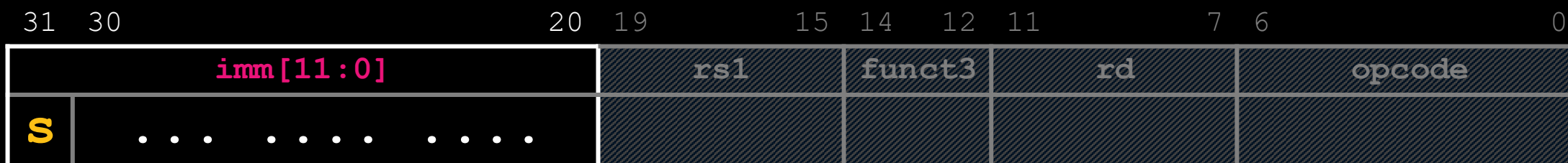
I-Type Immediate

GOT TO HERE



Instruction

inst[31:0]



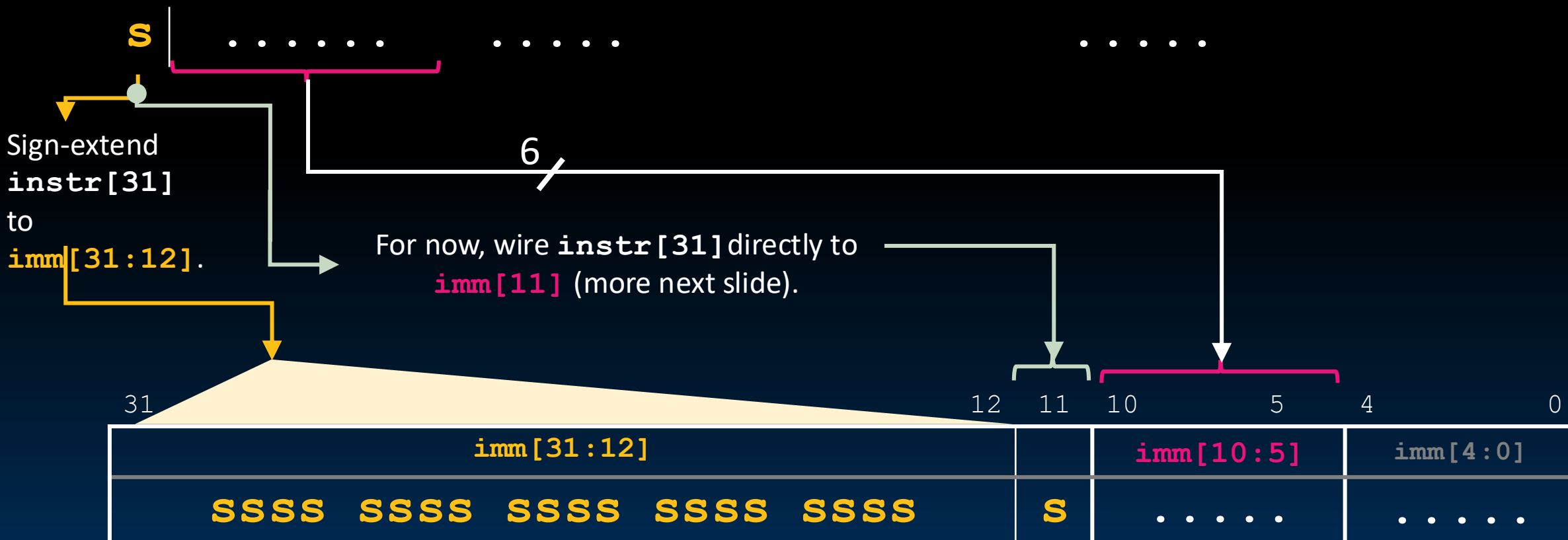
Copy upper 12 bits of instruction, inst[31:20], to lower 12 bits of immediate, imm[11:0].

Immediate

imm[31:0]

Sign-extend: Copy inst[31] to upper 20 bits of immediate, imm[31:12].







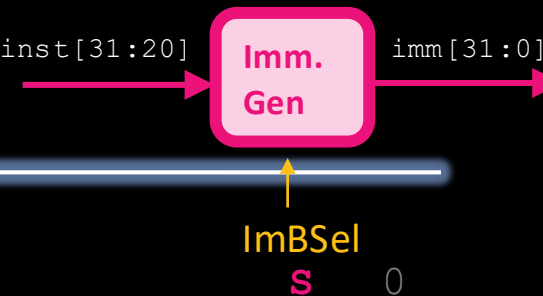
	31	30	25	24	20	19	15	14	12	11	7	6	S	0
I-Type	imm[11 10:5]				imm[4:0]		rs1		funct3		rd		I-OPCODE	
S-Type	imm[11 10:5]				rs2		rs1		funct3		imm[4:0]		S-OPCODE	



Immediate imm[31:0]



B-Type immediates, etc.



Instruction `inst[31:0]`

	31	30		25	24		20	19		15	14	12	11		7	6			0
I-Type	imm[11:5]					imm[4:0]					rs1	funct3	rd			I-OPCODE			
S-Type	imm[11 10:5]					rs2					rs1	funct3	imm[4:0]			S-OPCODE			
B-Type	imm[12 10:5]													imm[4:1 11]			B-OPCODE		
	S																		

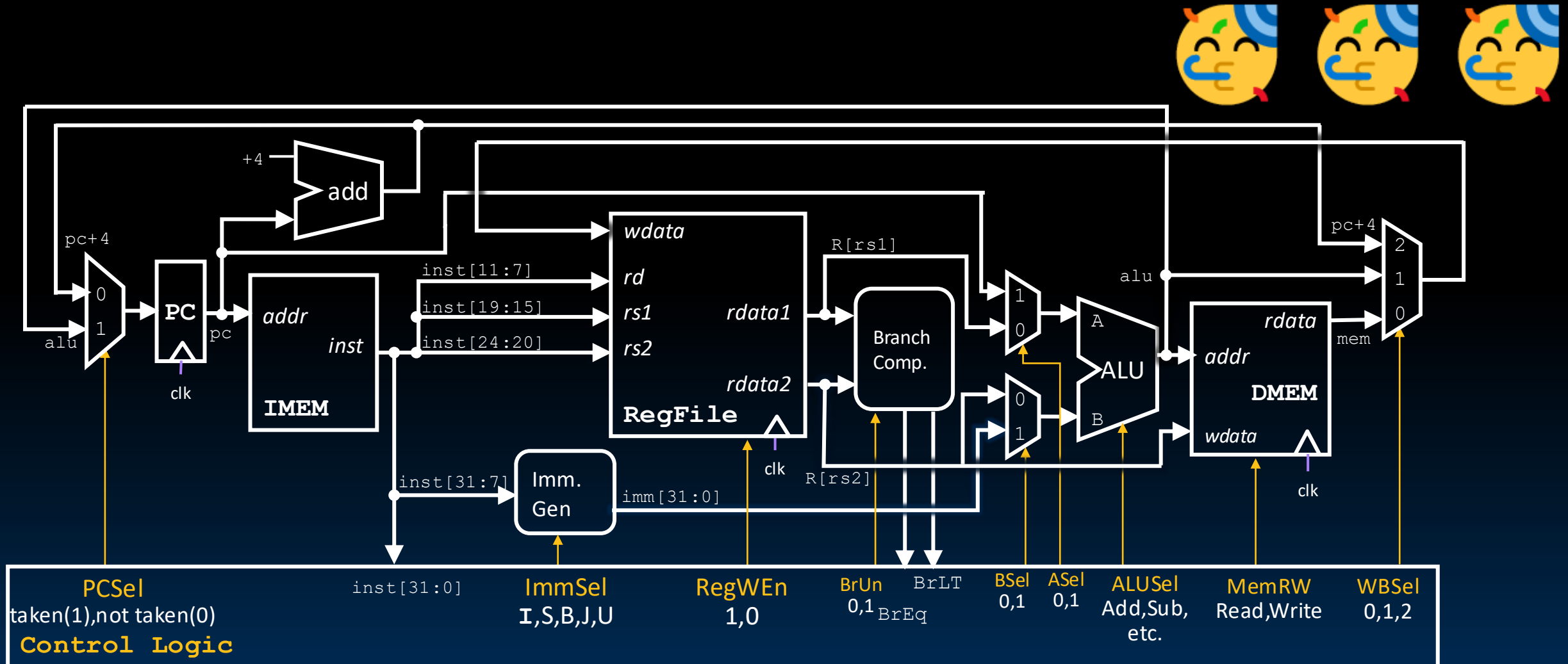
`instr[31]` is always the sign bit.

- MUX for `imm[11]`:
- S: `instr[31]`
 - B: `instr[7]`

- MUX for `imm[0]`:
- S: `instr[7]`
 - B: 0 (implicit 0; half-words → bytes)

31												12	11	10		5	4		1	0
imm[31:12]																imm[10:5]			imm[4:0]	
SSSS SSSS SSSS SSSS SSSS												.				.	.	.	.	.

Complete RV32I Datapath!



[at home] Jumps, U-Type

- Practice: R-, I-, S-Type
- Adding B-Type (Branches)
- Designing the Immediate Generation Block (cont.)
- [at home] Jumps, U-Type

j alr (J-Type), jal (I-Type), U-Type: Updates

Needs to support J- and U-Types:
two 20-bit immediate formats

Jumps: To save $rd = PC + 4$,
WBSel now controls
a 3-input MUX.

